

HTML - Cheat Sheet

HTML क्या है?

HTML का फुल फॉर्म HyperText Markup Language होता है। ये वेब पेज बनाने की सबसे बेसिक भाषा है। इसका काम होता है किसी भी वेब पेज का structure और content तय करना, मतलब कहाँ Heading आएगी, कहाँ Paragraph होगा, कहाँ Image लगेगी और कहाँ Link दिया जाएगा।

सीधी भाषा में कहें तो HTML वेबसाइट का ढांचा (Skeleton) तैयार करता है। इसके ऊपर फिर CSS से डिजाइन और JavaScript से functionality जोड़ी जाती है।

एक बात ध्यान देने वाली है, HTML कोई programming language नहीं है, ये एक markup language है। मतलब इसमें कोई logic या calculation नहीं होता। हम बस content के चारों तरफ tags लगाते हैं और browser उन tags को पढ़कर पेज बना देता है।

छोटा सा example:

```
<h1>मेरा नाम प्रिया है</h1>
<p>मैं एक front-end developer हूँ</p>
```

यहाँ `<h1>` browser को बता रहा है कि ये बड़ी heading है, और `<p>` बता रहा है कि ये एक paragraph है।

`<!DOCTYPE html>` क्या करता है?

`<!DOCTYPE html>` browser को बताता है कि ये document HTML5 में लिखा गया है। ये कोई HTML tag नहीं है, ये बस एक instruction है जो हमेशा file की सबसे ऊपर वाली line में आता है।

इसका असली फायदा ये है कि browser **standards mode** में page render करता है। अगर ये न लगाएँ तो browser **quirks mode** में चला जाता है, जहाँ पुराने जमाने के हिसाब से page render होता है और layout टूट सकता है।

```
<!DOCTYPE html>
<html>
  ...
</html>
```

एक HTML document के तीन मुख्य हिस्से कौन से हैं?

किसी भी HTML document में तीन मुख्य हिस्से होते हैं:

- `<!DOCTYPE html>` - सबसे ऊपर, browser को document type बताता है।
- `<head>` - page की meta information रखता है (title, meta tags, CSS links) जो user को सीधे दिखती नहीं।

3. **<body>** - असली content जो user को screen पर दिखता है (heading, paragraph, image वगैरह)।

```
<!DOCTYPE html>
<html>
  <head>
    <title>मेरा पेज</title>
  </head>
  <body>
    <h1>नमस्ते</h1>
  </body>
</html>
```

<head> और **<body>** में क्या फर्क है?

सीधा सा फर्क है:

- **<head>** में page की **meta information** होती है, जो user को सीधे दिखती नहीं। जैसे page का title, meta tags, CSS और JavaScript के links. ये browser और search engine के लिए होती है।
- **<body>** में वो सारा **visible content** होता है जो user screen पर देखता है, जैसे heading, paragraph, image, button वगैरह।

आसान शब्दों में, **<head>** background की setup है और **<body>** असली दिखने वाला page है।

<title> का इस्तेमाल क्यों करते हैं?

<title> tag page का नाम तय करता है जो **browser के tab** पर दिखता है। ये **<head>** के अंदर लिखा जाता है।

इसके दो बड़े फायदे हैं: - User को tab देखकर पता चलता है कि कौन सा page खुला है। - **SEO** के लिए ज़रूरी है, क्योंकि Google search results में यही title दिखता है।

```
<head>
  <title>प्रिया का Portfolio</title>
</head>
```

Meta tags का purpose क्या है?

Meta tags **<head>** के अंदर लगते हैं और page के बारे में **extra information** (metadata) देते हैं। ये user को दिखती नहीं, पर browser और search engine के लिए बहुत काम की होती है।

कुछ आम meta tags:

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="ये प्रिया का portfolio है">
```

- **charset** - कौन सा character encoding इस्तेमाल होगा (UTF-8 से सब भाषाओं के अक्षर सही दिखते हैं)।
- **viewport** - page mobile पर सही से responsive दिखे, इसके लिए ज़रूरी है।
- **description** - search engine results में page का छोटा सा description दिखाता है।

Block elements और inline elements में क्या फर्क है?

- **Block elements** हमेशा नई line से शुरू होते हैं और पूरी width ले लेते हैं। जैसे `<div>`, `<p>`, `<h1>`, `<ul>` ।
- **Inline elements** नई line नहीं लेते, बस उतनी ही जगह घेरते हैं जितना content है, और एक ही line में साथ-साथ बैठ जाते हैं। जैसे `<span>`, `<a>`, `<img>`, `<strong>` ।

```
<p>ये एक block है</p>      <!-- पूरी line ले लेगा -->
<span>ये inline है</span> <span>ये भी इसी line में</span>
```

छोटा सा फर्क और - block elements पर हम width/height set कर सकते हैं, inline पर सीधे नहीं होता।

Semantic elements क्या होते हैं?

Semantic elements वो tags हैं जिनके नाम से ही पता चल जाता है कि उनके अंदर किस तरह का content है। मतलब tag खुद अपना मतलब बता देता है।

जैसे `<header>` देखते ही समझ आ जाता है कि ये page का ऊपर वाला हिस्सा है। इसके उलट `<div>` एक non-semantic tag है, उसको देखकर पता नहीं चलता अंदर क्या है।

कुछ semantic HTML tags के नाम बताइए।

कुछ आम semantic tags:

- `<header>` - page या section का ऊपर वाला हिस्सा
- `<nav>` - navigation links
- `<main>` - page का मुख्य content
- `<section>` - content का एक logical हिस्सा
- `<article>` - अपने आप में पूरा एक content piece (जैसे blog post)
- `<aside>` - side का extra content (जैसे ads, sidebar)
- `<footer>` - page का सबसे नीचे वाला हिस्सा

Semantic HTML क्यों ज़रूरी है?

तीन बड़े कारण हैं:

1. **Readability** - code पढ़ने में आसान हो जाता है, दूसरे developer को तुरंत समझ आ जाता है कि कौन सा हिस्सा क्या है।
2. **SEO** - search engine semantic tags से page का structure अच्छे से समझता है, जिससे ranking में फायदा होता है।
3. **Accessibility** - screen readers (जो blind users के लिए होते हैं) semantic tags से page को सही तरीके से पढ़ पाते हैं।

मतलब `<div>` के ढेर लगाने से अच्छा है proper semantic tags use करना।

Table बनाने के लिए कौन से tags use होते हैं? एक table example दीजिए और हर tag समझाइए।

Table बनाने के लिए ये tags use होते हैं:

```
<table>
  <thead>
    <tr>
      <th>नाम</th>
      <th>उम्र</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>अजय</td>
      <td>24</td>
    </tr>
    <tr>
      <td>आदित्य</td>
      <td>26</td>
    </tr>
  </tbody>
</table>
```

हर tag का मतलब:

- `<table>` - पूरी table को wrap करता है।
 - `<thead>` - table का header वाला हिस्सा (column के नाम)।
 - `<tbody>` - table का असली data वाला हिस्सा।
 - `<tr>` - table row, यानी एक पूरी पंक्ति।
 - `<th>` - table heading cell, इसका text bold और center में आता है।
 - `<td>` - table data cell, इसमें असली data रखते हैं।
-

Tables कब use करनी चाहिए?

Table सिर्फ **tabular data** दिखाने के लिए use करनी चाहिए, यानी जो data rows और columns में natural तरीके से बैठता हो। जैसे price list, marksheet, comparison chart, report data वगैरह।

एक गलती जो पुराने जमाने में होती थी - लोग **page का layout** बनाने के लिए table use करते थे। ये अब बिल्कुल नहीं करना चाहिए। Layout के लिए CSS (Flexbox, Grid) use होता है, table सिर्फ data के लिए।

Image दिखाने के लिए कौन सा tag use होता है?

Image दिखाने के लिए `<img>` tag use होता है। ये एक self-closing tag है, मतलब इसका कोई closing tag नहीं होता।

```

```

- **src** - image कहाँ रखी है, उसका path या URL।
- **alt** - image का text description।
- **width / height** - image का size।

alt attribute क्यों ज़रूरी है?

alt attribute तीन वजह से ज़रूरी है:

1. **Image load न हो** तो उसकी जगह ये text दिख जाता है, ताकि user को पता रहे वहाँ क्या होना चाहिए था।
2. **Accessibility** - screen reader इसी **alt** text को पढ़कर blind users को बताता है कि image में क्या है।
3. **SEO** - search engine image का content **alt** से समझता है, क्योंकि वो खुद तो image “देख” नहीं सकता।

```

```

Media के लिए कौन से HTML tags use होते हैं?

Media दिखाने के लिए ये tags use होते हैं:

- **** - image के लिए।
- **<audio>** - sound या music चलाने के लिए।
- **<video>** - video चलाने के लिए।
- **<source>** - audio/video के अंदर अलग-अलग file formats देने के लिए।
- **<iframe>** - किसी दूसरे page या YouTube video को embed करने के लिए।

```
<video controls width="300">
  <source src="movie.mp4" type="video/mp4">
</video>
```

यहाँ `controls` attribute play/pause के buttons दिखा देता है।

Form बनाने के लिए कौन सा tag use होता है?

Form बनाने के लिए `<form>` tag use होता है। इसके अंदर हम input fields, buttons वगैरह रखते हैं जिससे user data भर सके।

```
<form action="/submit" method="POST">
  <input type="text" name="naam">
  <button type="submit">भेजें</button>
</form>
```

- `action` - data कहाँ भेजना है (server का URL)।
 - `method` - data कैसे भेजना है (GET या POST)।
-

GET और POST में क्या फर्क है?

दोनों form data भेजने के तरीके हैं, पर फर्क है:

- **GET** - data **URL में** दिखता है (`?naam=priya`)। इसलिए ये कम secure है। इसका use तब करते हैं जब data sensitive न हो, जैसे search query. Data की limit भी होती है।
- **POST** - data **request body में** छुपा कर भेजा जाता है, URL में नहीं दिखता। इसलिए ये ज्यादा secure है। Login, payment जैसे sensitive data के लिए POST use करते हैं। Data की limit नहीं होती।

छोटा सा रूल - data सिर्फ पढ़ना/लाना है तो GET, data बदलना/भेजना है तो POST।

Checkbox और radio button में क्या फर्क है?

- **Checkbox** - इसमें user **एक से ज्यादा options** select कर सकता है। जैसे “अपनी hobbies चुनो”।
- **Radio button** - इसमें एक group में से सिर्फ **एक ही option** select हो सकता है। जैसे “gender चुनो”।

```
<!-- Checkbox: कई चुन सकते हैं -->
<input type="checkbox" name="hobby"> Cricket
<input type="checkbox" name="hobby"> Music

<!-- Radio: एक ही चुन सकते हैं (same name होना ज़रूरी) -->
<input type="radio" name="gender"> पुरुष
<input type="radio" name="gender"> महिला
```

ध्यान रहे - radio buttons का `name` same रखना ज़रूरी है, तभी वो एक group बनते हैं।

Textarea किस लिए use होता है?

`<textarea>` का use **multi-line text** लेने के लिए होता है। मतलब जहाँ user को लंबा text लिखना हो, जैसे comment, message, address या feedback.

`<input type="text">` सिर्फ एक line लेता है, जबकि `<textarea>` कई lines ले सकता है।

```
<textarea rows="4" cols="30">यहाँ लिखें</textarea>
```

`rows` और `cols` से उसका size तय होता है।

हर जगह `<div>` use करने से क्यों बचना चाहिए?

`<div>` एक **non-semantic** tag है, उसको देखकर पता नहीं चलता अंदर क्या content है। अगर पूरे page में सिर्फ `<div>` ही `<div>` भर दें (जिसको “div soup” कहते हैं) तो तीन नुकसान होते हैं:

1. Code पढ़ने में मुश्किल हो जाता है।
2. **SEO** खराब होता है, search engine structure नहीं समझ पाता।
3. **Accessibility** टूट जाती है, screen reader को समझ नहीं आता कौन सा हिस्सा क्या है।

इसकी जगह जहाँ possible हो वहाँ semantic tags (`<header>` , `<nav>` , `<section>`) use करने चाहिए। `<div>` सिर्फ तब use करें जब कोई semantic tag fit न बैठे।

Meta description का purpose क्या है?

Meta description एक meta tag है जो page का **छोटा सा summary** देता है। ये search engine results में page के title के नीचे दिखता है।

```
<meta name="description" content="प्रिया का front-end portfolio, projects और skills देखें।">
```

अच्छा description लिखने से user को search results में page का मतलब समझ आता है और click करने का chance बढ़ जाता है। ये indirectly **SEO और CTR** दोनों में मदद करता है।

Semantic HTML SEO में कैसे मदद करता है?

Semantic HTML से search engine को page का **structure साफ-साफ समझ आता है** - कौन सा हिस्सा heading है, कौन सा main content है, कौन सा navigation है।

जब Google को structure अच्छे से समझ आता है तो वो page को सही keywords पर index करता है, जिससे ranking बेहतर होती है।

`<div>` soup में Google को सब एक जैसा दिखता है, इसलिए वो content की importance नहीं समझ पाता।

क्या एक page में कई `<h1>` tags होने चाहिए?

आम तौर पर एक page में **एक ही `<h1>`** होना चाहिए, जो page का main title हो। इससे search engine और screen reader दोनों को साफ पता चलता है कि page किस बारे में है।

HTML5 में technically कई `<h1>` allowed हैं (हर section में एक), पर SEO और clarity के लिए **एक ही `<h1>` रखना** best practice मानी जाती है।

क्या एक page में कई `<h2>`, `<h3>`, `<h4>` tags हो सकते हैं?

हाँ, इनको कई बार use कर सकते हैं। ये sub-headings हैं और page में जितने sections/sub-sections हों उतनी बार आ सकती हैं।

बस एक rule है - heading का **order सही रखें**। `<h1>` के बाद `<h2>`, उसके बाद `<h3>`। बीच में skip न करें (जैसे `<h1>` के तुरंत बाद `<h4>`)। इससे page की hierarchy साफ रहती है।

DevTools क्या है? हम इसे क्यों use करते हैं?

DevTools (Developer Tools) browser में built-in एक tool है (Chrome, Firefox सबमें होता है)। इसको खोलने के लिए **F12** या right-click पर “Inspect” दबाते हैं।

इसका use हम इन चीज़ों के लिए करते हैं: - HTML और CSS को **live देखने और बदलने** के लिए। - **Bug ढूँढने** (debugging) के लिए। - JavaScript errors console में देखने के लिए। - Page कितनी जल्दी load हो रहा है, network requests कैसी जा रही हैं - ये देखने के लिए।

Developer के लिए ये रोज़ का सबसे काम का tool है।

DevTools में कौन सी information देख सकते हैं?

DevTools में कई tabs होते हैं, हर एक अलग जानकारी देता है:

- **Elements** - page का HTML structure और लगी हुई CSS। यहाँ live edit भी कर सकते हैं।
- **Console** - JavaScript errors, warnings और `console.log` के outputs।
- **Network** - कौन सी files/API requests जा रही हैं, कितना time लग रहा है, status code क्या है।
- **Sources** - page की saari files (JS, CSS), यहाँ breakpoints लगाकर debug करते हैं।
- **Application** - cookies, localStorage, sessionStorage जैसी stored चीज़ें।
- **Performance** - page load और speed की detailed report।

आम तौर पर front-end काम में Elements, Console और Network सबसे ज़्यादा use होते हैं।

जब कोई image नहीं दिख रही हो तो आप क्या करते हैं? क्या-क्या वजहें हो सकती हैं?

सबसे पहले मैं **DevTools का Console और Network tab** खोलकर देखती हूँ कि image की request fail तो नहीं हुई। आम वजहें ये होती हैं:

1. **गलत `src` path** - file का नाम या folder का रास्ता गलत है (सबसे आम वजह)।

2. **File name की spelling या case गलत** - server पर `Photo.jpg` और `photo.jpg` अलग होते हैं।
3. **File असल में मौजूद ही नहीं** या delete हो गई।
4. **Image का URL टूटा हुआ** (अगर external link है)।
5. **Extension गलत** - file `.png` है पर `src` में `.jpg` लिखा है।
6. **Network/permission issue** - file access नहीं हो पा रही (403/404)।

`alt` text लगा हो तो कम से कम user को पता चल जाता है वहाँ क्या होना चाहिए था।

HTML में CSS कैसे जोड़ते हैं?

CSS जोड़ने के तीन तरीके हैं:

1. **Inline CSS** - सीधे tag के अंदर `style` attribute में।

```
<p style="color: red;">लाल text</p>
```

2. **Internal CSS** - `<head>` के अंदर `<style>` tag में।

```
<style>
p { color: red; }
</style>
```

3. **External CSS** - अलग `.css` file बनाकर `<link>` से जोड़ना।

```
<link rel="stylesheet" href="style.css">
```

CSS जोड़ने का कौन सा तरीका best है और क्यों?

External CSS सबसे अच्छा तरीका माना जाता है। वजहें:

1. **Reusability** - एक ही CSS file कई pages में use कर सकते हैं।
2. **Separation of concerns** - HTML (structure) और CSS (design) अलग-अलग रहते हैं, code साफ रहता है।
3. **Caching** - browser CSS file को cache कर लेता है, जिससे page तेज़ load होता है।
4. **Maintain करना आसान** - एक जगह change करो, सब pages पर असर हो जाता है।

Inline CSS सबसे खराब, क्योंकि वो हर tag पर अलग लगती है और reuse नहीं होती।

Responsive design का क्या मतलब है?

Responsive design का मतलब है website हर screen size पर **सही से दिखे और काम करे** - चाहे mobile हो, tablet हो या बड़ा desktop.

इसके लिए हम CSS में **media queries**, flexible units (`%` , `rem` , `vw`) और Flexbox/Grid use करते हैं। साथ में viewport meta tag भी ज़रूरी है:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

आसान शब्दों में - एक ही website mobile पर भी टूटे बिना अच्छी दिखे।

HTML में JavaScript कैसे जोड़ते हैं?

JavaScript जोड़ने के दो तरीके हैं:

1. **Internal** - सीधे `<script>` tag के अंदर code लिखना।

```
<script>
  console.log("नमस्ते");
</script>
```

2. **External** - अलग `.js` file बनाकर जोड़ना (ये preferred है)।

```
<script src="app.js"></script>
```

External तरीका बेहतर है क्योंकि code reuse होता है और HTML साफ रहता है।

Scripts को आमतौर पर नीचे क्यों रखते हैं या `defer` से क्यों load करते हैं?

जब browser HTML पढ़ते-पढ़ते बीच में `<script>` देखता है, तो वो रुक जाता है, पहले script download और run करता है, फिर आगे बढ़ता है। इससे page की दिखने की speed slow हो जाती है।

इसका हल: - **Script को `<body>` के सबसे नीचे** रखना, ताकि पहले पूरा content दिख जाए, फिर script चले। - या `defer` attribute use करना, जिससे script HTML के साथ-साथ download हो जाती है पर run तब होती है जब पूरा HTML तैयार हो।

```
<script src="app.js" defer></script>
```

एक और फायदा - script को तब तक की HTML elements मिल जाते हैं, तो “element not found” वाली error नहीं आती।

Inline styles से क्यों बचना चाहिए?

तीन वजह से inline styles से बचना चाहिए:

1. **Reuse नहीं होता** - हर tag पर अलग-अलग style लिखनी पड़ती है, code repeat होता है।
2. **Maintain करना मुश्किल** - color बदलना हो तो हर जगह जाकर बदलना पड़ता है।
3. **HTML गंदा हो जाता है** - structure और design मिल जाते हैं, पढ़ना मुश्किल।

साथ ही inline style की **specificity** बहुत ज़्यादा होती है, इसलिए external CSS से उसे override करना सिरदर्द बन जाता है।

File names और class तथा id names meaningful क्यों होने चाहिए?

क्योंकि code सिर्फ एक बार नहीं लिखा जाता, उसको बार-बार पढ़ा और maintain किया जाता है। Meaningful नाम से:

- **खुद को और team को** तुरंत समझ आता है कि वो चीज़ क्या करती है।
- नया developer आए तो उसको समझने में time नहीं लगता।
- Debugging आसान हो जाती है।

जैसे `class="red-box"` से अच्छा है `class="error-message"` . और `div1` , `xyz` , `test123` जैसे नाम बिल्कुल नहीं रखने चाहिए, इनका कोई मतलब नहीं निकलता।

Images को optimize क्यों करना चाहिए?

Image optimize करने का मतलब है उसका size कम रखना बिना quality ज़्यादा खराब किए। वजहें:

1. **Page तेज़ load होता है** - बड़ी images सबसे ज़्यादा page slow करती हैं।
2. **कम data खर्च** होता है, खासकर mobile users के लिए।
3. **SEO बेहतर** - Google fast pages को ज़्यादा पसंद करता है।
4. **अच्छा user experience** - कोई page load होने का इंतज़ार करना पसंद नहीं करता।

इसके लिए सही format (jpg/webp), सही size, और compression use करते हैं।

Lazy loading क्या है?

Lazy loading का मतलब है किसी चीज़ को **तभी load करना जब उसकी ज़रूरत हो**, शुरू में ही सब कुछ एक साथ load न करना।

जैसे एक page पर 50 images हैं। बिना lazy loading के browser शुरू में ही सारी 50 images download कर लेगा, चाहे user नीचे scroll करे या न करे। इससे page slow हो जाता है। Lazy loading में सिर्फ वही images load होती हैं जो screen पर दिख रही हैं, बाकी तब load होती हैं जब user scroll करके उन तक पहुँचता है।

फायदा - page पहली बार **तेज़ load** होता है और बेकार का data खर्च नहीं होता।

Lazy loading कैसे implement करते हैं?

सबसे आसान तरीका है HTML में `loading="lazy"` attribute लगाना। ये images और iframes दोनों पर काम करता है:

```

<iframe src="video.html" loading="lazy"></iframe>
```

इतना लगाते ही browser खुद वो image तब तक load नहीं करता जब तक user उसके पास scroll न कर ले। ये native तरीका है, इसमें कोई extra library नहीं चाहिए।

`loading` attribute की तीन values होती हैं: - `lazy` - ज़रूरत पड़ने पर load करो। - `eager` - तुरंत load करो (default behaviour जैसा)। - `auto` - browser खुद decide करे।

एक ध्यान देने वाली बात - जो images शुरू में screen पर दिख रही हों (जैसे top banner), उन पर lazy मत लगाओ, वरना वो देर से दिखेंगी और experience खराब होगा।

पुराने या ज्यादा control वाले cases में JavaScript का **Intersection Observer API** भी use करते हैं, जो detect करता है कि element screen में आया या नहीं, और तभी image load करता है।

`<b>` और `<strong>` में क्या फर्क है?

देखने में दोनों text को **bold** कर देते हैं, पर असली फर्क मतलब (semantics) का है:

- `<b>` सिर्फ text को bold दिखाता है, इसका कोई खास मतलब नहीं होता। ये बस visual है।
- `<strong>` बताता है कि ये text **important** है। Screen reader इसको ज़ोर देकर पढ़ता है और search engine भी इसको ज्यादा weight देता है।

मतलब अगर सिर्फ दिखावे के लिए bold चाहिए तो `<b>`, पर अगर text सच में ज़रूरी है (जैसे warning, key point) तो `<strong>` use करना चाहिए।

```
<b>ये बस bold है</b>
<strong>ये ज़रूरी बात है</strong>
```

इसी तरह `<i>` (सिर्फ italic) और `<em>` (emphasis, मतलब वाला italic) का भी फर्क होता है।

`href` attribute क्या करता है?

`href` (Hypertext REference) `<a>` tag में बताता है कि link **कहाँ ले जाएगा**, यानी उसका destination क्या है। ये किसी दूसरे page, website, file या same page के किसी section का address हो सकता है।

```
<a href="https://google.com">Google खोलो</a>
<a href="about.html">About page</a>
<a href="#contact">नीचे contact section पर जाओ</a>
```

बिना `href` के `<a>` tag सिर्फ एक normal text रह जाता है, उस पर click करने का कोई फायदा नहीं होता।

किसी link को नए tab में कैसे खोलते हैं?

`<a>` tag में `target="_blank"` लगाने से link **नए tab** में खुलता है।

```
<a href="https://google.com" target="_blank" rel="noopener noreferrer">Google</a>
```

एक ज़रूरी best practice - साथ में `rel="noopener noreferrer"` भी लगाना चाहिए। ये एक security issue (जिसको "tabnabbing" कहते हैं) से बचाता है, जहाँ नया खुला page आपके original page को control कर सकता है।

Absolute और relative URLs में क्या फर्क है?

- **Absolute URL** - पूरा address देता है, domain के साथ। ये कहीं से भी same जगह ले जाएगा। आम तौर पर दूसरी websites के लिए use होता है।

```
<a href="https://example.com/about.html">About</a>
```

- **Relative URL** - सिर्फ current page के हिसाब से रास्ता देता है, domain नहीं लिखते। ये अपनी ही website के अंदर navigate करने के लिए use होता है।

```
<a href="about.html">About</a>
<a href="../images/logo.png">Logo</a>
```

फायदा - relative URL से website को कहीं भी move करो (localhost से production), links टूटते नहीं, क्योंकि वो domain पर depend नहीं करते।

PNG, JPEG, WEBP, GIF और SVG में क्या फर्क है? किस situation में क्या prefer करें?

हर format का अपना काम है:

- **JPEG (.jpg)** - photos के लिए best. File size छोटा रखता है पर transparency support नहीं करता। जहाँ रंग ज़्यादा हों (असली photos) वहाँ use करो।
- **PNG** - transparency support करता है और quality अच्छी रहती है, पर file size बड़ा होता है। Logo, icons, या जहाँ transparent background चाहिए वहाँ use करो।
- **WEBP** - modern format, JPEG और PNG दोनों से छोटा size देता है साथ में अच्छी quality और transparency भी। आजकल web के लिए सबसे अच्छा माना जाता है, बस बहुत पुराने browsers में issue हो सकता है।
- **GIF** - simple animations के लिए (छोटे moving images). सिर्फ 256 रंग support करता है, इसलिए photos के लिए ठीक नहीं।
- **SVG** - ये pixel-based नहीं, **vector** (math-based) होता है, इसलिए कितना भी zoom करो quality खराब नहीं होती। Logo, icons, illustrations के लिए perfect, क्योंकि हर screen size पर sharp दिखता है।

सीधा rule - photo है तो **JPEG/WEBP**, transparency चाहिए तो **PNG/WEBP**, logo/icon है तो **SVG**, छोटी animation है तो **GIF** (या बेहतर video).

क्या एक page में कई header elements हो सकते हैं?

हाँ, हो सकते हैं। ज़्यादातर लोग सोचते हैं `<header>` सिर्फ एक बार, page के सबसे ऊपर आता है, पर ऐसा नहीं है।

`<header>` का मतलब है किसी **section का ऊपर वाला हिस्सा**। तो page में एक main `<header>` हो सकता है, और साथ ही हर `<article>` या `<section>` का अपना अलग `<header>` भी हो सकता है।

```
<header>पूरे page का header</header>

<article>
  <header>इस article का header</header>
  <p>article का content...</p>
</article>
```

बस ध्यान रहे, ये एक-दूसरे के अंदर nested न हों गलत तरीके से।

अगर CSS load न हो तो क्या होता है?

Page टूटता नहीं, बस उसका **design/styling गायब** हो जाता है। Content तो पूरा दिखता है क्योंकि वो HTML में होता है, पर वो बिना किसी layout, color, या spacing के plain दिखता है - जैसे एक सादा text document.

इसी वजह से कहते हैं कि page का structure HTML में सही होना चाहिए, ताकि CSS fail भी हो जाए तो content readable रहे। ये **graceful degradation** कहलाता है।

अगर JavaScript load न हो तो क्या होता है?

ये इस बात पर depend करता है कि page JavaScript पर कितना depend करता है:

- अगर page का content HTML में पहले से है, तो content दिखता रहेगा, बस **interactive चीज़ें** (jaise dropdown, slider, form validation, button clicks) काम नहीं करेंगी।
- अगर पूरा page ही JavaScript से बनता है (जैसे कुछ React/SPA apps), तो हो सकता है **खाली page** या कुछ भी न दिखे।

इसलिए अच्छी practice है ज़रूरी content HTML में रखना और JavaScript को सिर्फ extra functionality के लिए use करना। इसको **progressive enhancement** कहते हैं - पहले basic चीज़ काम करे, फिर ऊपर से JS से features जोड़ो।

Favicon क्या है?

Favicon वो छोटा सा icon है जो browser के **tab में page के title के बगल** में दिखता है। Bookmark करने पर और history में भी यही icon दिखता है।

ये `<head>` में `<link>` से जोड़ा जाता है:

```
<link rel="icon" href="favicon.ico" type="image/x-icon">
```

छोटी चीज़ है पर ज़रूरी है, इससे website professional और पहचानने लायक लगती है। जैसे tab में Google का “G” या YouTube का लाल icon - वो favicon ही है।

Iframe किस लिए use होता है?

`<iframe>` (inline frame) का use किसी दूसरे page या content को अपने page के अंदर embed करने के लिए होता है। मतलब एक page के अंदर दूसरा page दिखाना।

```
<iframe src="https://youtube.com/embed/abc123" width="560" height="315"></iframe>
```

आम use cases: - YouTube video embed करना। - Google Maps दिखाना। - किसी payment gateway या third-party widget को load करना।

एक ध्यान देने वाली बात - iframe security और performance के मामले में थोड़ा भारी होता है, इसलिए सिर्फ जरूरत पर use करना चाहिए, और external content के लिए `sandbox` attribute लगाना safe रहता है।

एक modern front-end project का structure कैसा होता है?

Project के हिसाब से थोड़ा बदलता है, पर एक basic साफ-सुथरा structure ऐसा होता है - सबसे बड़ी बात ये कि files **type के हिसाब से अलग folders** में रखी जाती हैं, सब कुछ एक जगह नहीं डाला जाता:

```
my-project/
├─ index.html
├─ assets/
│  ├─ images/      # सारी images
│  ├─ fonts/       # custom fonts
│  └─ icons/       # icons, favicon
├─ css/
│  └─ style.css    # styles
├─ js/
│  └─ app.js       # JavaScript
├─ pages/          # बाकी HTML pages (about.html वगैरह)
└─ README.md      # project की जानकारी
```

मुख्य बातें: - `index.html` हमेशा root में, क्योंकि यही entry point होता है। - **CSS, JS, images अलग-अलग folders** में, ताकि code साफ और ढूंढने में आसान रहे। - **Meaningful folder names** ताकि नए developer को तुरंत समझ आ जाए कहाँ क्या है।

बड़े modern projects (React, Angular जैसे frameworks) में structure और detailed होता है - `components/`, `services/`, `utils/` जैसे folders आ जाते हैं, और build tool (जैसे Vite/Webpack) final code को `dist/` folder में bundle कर देता है। पर सोच वही रहती है - **हर चीज़ अपनी सही जगह पर।**